

Activité 1 : le triangle de Pascal

1.1 Principe

En mathématiques, le triangle de Pascal est une présentation des coefficients binomiaux dans un triangle. Il fut nommé ainsi en l'honneur du mathématicien français Blaise Pascal. Il est connu sous l'appellation « triangle de Pascal » en Occident, bien qu'il fût étudié par d'autres mathématiciens, parfois plusieurs siècles avant lui.

Le triangle de Pascal

```

1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1

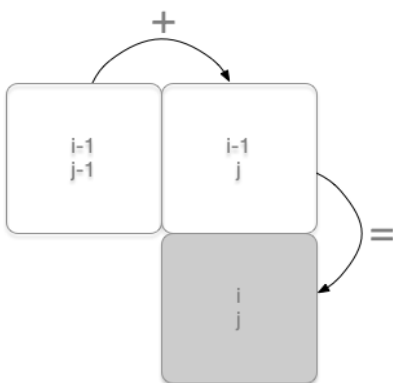
```

FIGURE 1 – premières lignes du triangle de Pascal

Cette figure permet de calculer les coefficients binomiaux d'un polynôme $(x+y)$ à la puissance n :

Un coefficient quelconque du triangle, situé à la ligne i et à la colonne j est calculé à partir de la formule de récurrence : (i et j supérieurs à 1)

$$C \binom{i}{j} = C \binom{i-1}{j-1} + C \binom{i-1}{j}$$



1.2 Algorithme récursif non optimisé

```

1 def pascal_recur(n,p):
2     if p==0: return 1
3     if p>n:
4         return 0
5     else:
6         return pascal_recur(n-1,p) + pascal_recur(n-1,p-1)

```

Question a : Evaluer la complexité asymptotique de cette fonction récursive.

On peut alors tester utiliser cette fonction avec le programme :

```

1 def pascal(n):
2     T = [[0] * (n+1) for p in range(n+1)]
3     for n in range(n+1):
4         for k in range(n+1):
5             T[n][k] = pascal_recur(n,k)
6     return T
7
8 > pascal(9)
9
10 [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
11  [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
12  [1, 2, 1, 0, 0, 0, 0, 0, 0, 0],
13  [1, 3, 3, 1, 0, 0, 0, 0, 0, 0],
14  [1, 4, 6, 4, 1, 0, 0, 0, 0, 0],
15  [1, 5, 10, 10, 5, 1, 0, 0, 0, 0],
16  [1, 6, 15, 20, 15, 6, 1, 0, 0, 0],
17  [1, 7, 21, 35, 35, 21, 7, 1, 0, 0],
18  [1, 8, 28, 56, 70, 56, 28, 8, 1, 0],
19  [1, 9, 36, 84, 126, 126, 84, 36, 9, 1]]

```

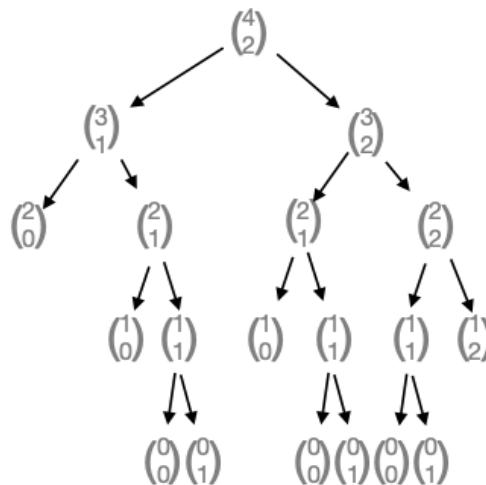


FIGURE 2 – arbre de calculs des coefficients binomiaux

Question b : Cet arbre, montre t-il une divergence exponentielle du nombre de calculs effectués ? Conclure.

Question c : Que pourrait-on faire, en rapport avec la programmation dynamique, pour réduire ce nombre de calculs ?

Le problème du rendu de monnaie

Etant donné un système de monnaie (pièces et billets), comment rendre une somme donnée de façon optimale, c'est-à-dire avec le nombre minimal de pièces et billets ?

Par exemple, la meilleure façon de rendre 7 euros est de rendre un billet de cinq et une pièce de deux, même si d'autres façons existent (rendre 7 pièces de un euro, par exemple).

Un système monétaire non canonique : Avant la décimalisation définitive de 1971, le système monétaire britannique reposé sur une subdivision de valeurs qui remontait à l'époque carolingienne.

Voici les principales valeurs de cet ancien système : 1 penny, 3 pence (pluriel de penny), 6 pence (ou demi-shilling), 12 pence (appelé shilling), 24 pence (appelé florin), 30 pence (appelé demi-couronne), 60 pence (appelé couronne), 240 pence (appelé livre)

2.1 algorithme naïf

Supposons que l'on dispose d'une fonction récursive, qui pour chaque pièce de la caisse fait le choix suivant :

- soit la pièce est inférieure à la monnaie à rendre : alors on soustrait la pièce à la somme à rendre et on appelle la fonction de manière récursive avec cette même caisse, et la nouvelle somme à rendre.
- soit la pièce est supérieure à la somme à rendre. on retire la pièce de la caisse. Et on appelle de manière récursive la fonction avec la nouvelle caisse, et la même somme.

Question d : Que renvoie la fonction pour rendre 53 pences avec le système imperial où $\text{pieces} = [240, 60, 30, 24, 12, 6, 3, 1]$? Y-a-t-il une meilleure combinaison que celle donnée par l'algorithme naïf ?

Question e : Représenter une partie de l'arbre des combinaisons possibles pour rendre un montant égal à 15, en choisissant une à une chaque pièce de la caisse $[1, 7, 9]$. Quel est le choix optimal ?

2.2 Algorithme glouton

Un algorithme **glouton** (*greedy* en anglais) fait un choix local immédiat et irrévocable à chaque étape, sans jamais revenir en arrière. Pour le rendu de monnaie, la stratégie gloutonne consiste à **toujours prendre la pièce de plus grande valeur possible**.

Exemple : rendre 15 avec $[1, 7, 9]$:

1. La plus grande pièce ≤ 15 est 9 $\rightarrow$ on la prend. Reste : 6.
2. La plus grande pièce ≤ 6 est 1 $\rightarrow$ on la prend. Reste : 5.
3. ... on prend encore 1, puis 1, puis 1, puis 1, puis 1.

Résultat : $9 + 1 + 1 + 1 + 1 + 1 + 1 = 7$ pièces.

Or la solution optimale est $7 + 7 + 1 = 3$ pièces. L'algorithme glouton n'est donc **pas toujours optimal** — il passe à côté de la bonne combinaison car il s'engage trop tôt sur la pièce de 9.

L'algorithme glouton fonctionne bien sur les systèmes monétaires "canoniques" (comme l'euro), mais échoue sur des systèmes non canoniques comme $[1, 7, 9]$ ou l'ancien système britannique.

2.3 Algorithme de programmation dynamique

La **programmation dynamique** adopte une approche différente : on construit un tableau de solutions optimales pour tous les montants de 0 à x , en s'appuyant à chaque étape sur les résultats déjà calculés.

Principe (approche ascendante) : pour rendre le montant w , il faut au moins une pièce, à choisir parmi les n pièces disponibles. Une fois cette pièce de valeur p choisie, le montant restant $w - p$ est strictement inférieur à w — on sait donc déjà le rendre de façon optimale (c'est déjà dans le tableau). Il suffit d'essayer les n possibilités et de retenir celle qui minimise le nombre total de pièces :

$$dp[w] = \min_{p \in \text{pièces}, p \leq w} (dp[w - p] + 1)$$

Exemple : rendre 15 avec des pièces de $[1, 7, 9]$.

Le tableau $dp[i][w]$ indique le nombre minimum de pièces pour rendre w en utilisant les i premiers types de pièces. ∞ signifie "impossible".

Ligne 0 — sans pièce :

- $dp[0][0] = 0$: rendre 0 ne nécessite aucune pièce
- $dp[0][w] = \infty$ pour tout $w > 0$: impossible sans pièce

Ligne 1 — pièces disponibles : $\{1\}$:

- $dp[1][w] = w$ pour tout w : il faut exactement w pièces de 1

Ligne 2 — pièces disponibles : $\{1, 7\}$:

- Pour $w < 7$: la pièce de 7 est inutilisable $\rightarrow$ on recopie la ligne 1
- Pour $w \geq 7$: on compare "ne pas prendre la pièce de 7" ($dp[1][w]$) et "la prendre" ($dp[2][w-7] + 1$) :
 - $w = 7$: $\min(7, dp[2][0]+1) = \min(7, 1) = 1$
 - $w = 8$: $\min(8, dp[2][1]+1) = \min(8, 2) = 2$
 - $w = 14$: $\min(14, dp[2][7]+1) = \min(14, 2) = 2$
 - $w = 15$: $\min(15, dp[2][8]+1) = \min(15, 3) = 3$

Note : quand on "prend" la pièce de 7, on consulte $dp[2][w-7]$ (et non $dp[1][w-7]$), car on peut réutiliser la même pièce plusieurs fois.

Ligne 3 — pièces disponibles : $\{1, 7, 9\}$:

- Pour $w < 9$: la pièce de 9 est inutilisable $\rightarrow$ on recopie la ligne 2
- Pour $w \geq 9$: on compare "ne pas prendre la pièce de 9" ($dp[2][w]$) et "la prendre" ($dp[3][w-9] + 1$) :
 - $w = 9$: $\min(dp[2][9], dp[3][0]+1) = \min(3, 1) = 1$
 - $w = 14$: $\min(dp[2][14], dp[3][5]+1) = \min(2, 6) = 2$
 - $w = 15$: $\min(dp[2][15], dp[3][6]+1) = \min(3, 7) = 3$

Tableau complet :

	w=0	w=1	w=2	...	w=6	w=7	w=8	w=9	w=10	w=11	w=12	w=13	w=14	w=15
sans pièce	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
+ pièce 1	0	1	2	...	6	7	8	9	10	11	12	13	14	15
+ pièce 7	0	1	2	...	6	1	2	3	4	5	6	7	2	3
+ pièce 9	0	1	2	...	6	1	2	1	2	3	4	5	2	3

La réponse se lit dans la dernière case : $dp[3][15] = 3$ pièces (7 + 7 + 1) — bien mieux que les 7 pièces du glouton !

Complexité : le tableau comporte $n \times W$ cases, chacune calculée en temps constant $\rightarrow O(n \cdot W)$.

Question f : En quoi l'approche par programmation dynamique réduit-elle le nombre de combinaisons à explorer par rapport à l'algorithme naïf ? Quel rôle joue le tableau dp dans cette réduction ?

Partie 3

Le problème du sac à dos

Un problème très similaire est celui du sac-à-dos. Supposons que vous découvriez la caverne d'Ali Baba (et des 40 voleurs), il y a une infinité d'objets de valeur $v_1, v_2, \dots, v_n$ mais chaque objet de valeur v_i a un poids de p_i . Comment remplir votre sac-à-dos avec le contenu qui rapportera le plus, sachant que votre sac ne supporte qu'un poids W ?

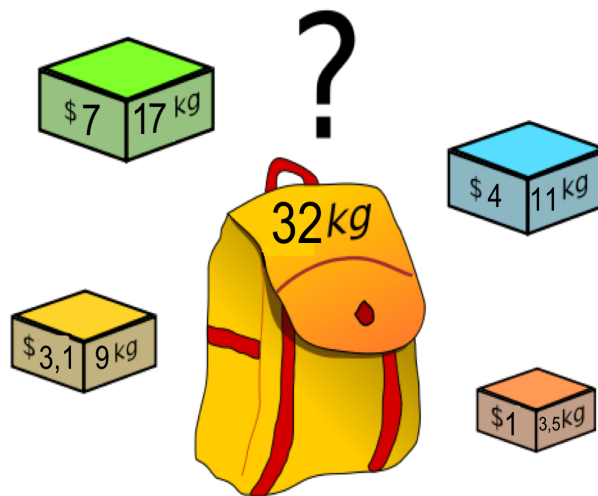


FIGURE 3 – probleme du sac a dos

3.1 Algorithme glouton

Une approche **gloutonne** pour le sac à dos consiste à trier les objets par **ratio valeur/poids décroissant**, puis à remplir le sac en prenant d'abord l'objet le plus "rentable" au kilo, jusqu'à ce qu'il ne rentre plus, puis le suivant, etc.

C'est simple et rapide, mais cette stratégie n'est **pas toujours optimale** : en s'engageant trop tôt sur l'objet le plus rentable, on peut bloquer de la capacité qui aurait permis une meilleure combinaison.

Remarque : dans la version fractionnaire du sac à dos (où on peut prendre une fraction d'un objet), l'algorithme glouton est en revanche toujours optimal. C'est uniquement la version entière (0/1) qui lui résiste.

3.2 Algorithme de programmation dynamique

Nous allons adapter l'algorithme de programmation dynamique du rendu de monnaie au problème du sac à dos. La structure est très similaire — c'est précisément ce qui fait la force de cette approche.

Structure de données :

rendu de monnaie	sac à dos
[Nbre de pieces, [pieces de 1, pieces de 2, pieces de 5, ...]]	[Gains cumulés, [articles 1, articles 2, articles 3, ...]]

Algorithme — tableau $dp[i][w]$ = valeur maximale atteignable avec les i premiers objets et un sac de capacité w :

rendu de monnaie	sac à dos
Pour toute valeur de montant i jusqu'à x	Pour tout poids i jusqu'à W
Pour tout montant de piece de la caisse	Pour tout objet (v_i, p_i) du trésor
Test 1 : le montant de la piece est-il $\leq i$?	Test 1 : Le poids de l'objet est-il $\leq i$?
Test 2 : Si j'ajoute la piece, le nombre de pieces rendues sera-t-il inférieur à celui stocké pour i ? (le reste a déjà été calculé)	Test 2 : Si je prends l'objet, le gain total sera-t-il supérieur à la valeur stockée pour i ? (le reste a déjà été calculé)
Si oui : rendre la piece	Si oui : prendre l'objet

La règle de remplissage s'écrit :

$$dp[i][w] = \max(dp[i-1][w], dp[i][w - p_i] + v_i) \quad \text{si } p_i \leq w$$

Note : on utilise $dp[i][w - p_i]$ (et non $dp[i-1][w - p_i]$) car les objets sont disponibles en nombre infini — on peut reprendre le même objet plusieurs fois.

Complexité : le tableau comporte $n \times W$ cases $\rightarrow O(n \cdot W)$.

Question g : Déterminer le choix idéal pour un sac à dos de capacité 32kg, avec les objets suivants, proposés en nombre infini : [{"valeur" : 7, "poids" : 13}, {"valeur" : 4, "poids" : 11}, {"valeur" : 3.1, "poids" : 9}, {"valeur" : 1, "poids" : 3.5}]

Objets	1	2	3	4
valeur	7	4	3.1	1
poids (kg)	17	11	9	3.5
ratio val/poids	0.41	0.36	0.34	0.29